API Legión+

Documentación de una API REST de películas

Juan Diego Velasco Quintero

Juan Camilo Calderón Delgado

Diana Marcela Ayala Ortiz

Maryori Brigith Barrera Ascanio

Laura Valentina Bautista Poveda

Docente: Ángel Manuel Rivera

Universidad Santo Tomás, Seccional Tunja

Ingeniería de Sistemas

Cloud Computing para el Desarrollo de Software

2026

Tabla de contenido

1. Introducción 4
2. Objetivos 4
    2.1. Objetivo general 4
    2.2. Objetivos específicos 4
3. Descripción de la API 4
4. Tecnologías y estructura del proyecto 5
    4.1. Tecnologías 5
    4.2. Archivos del proyecto 5
5. Base de datos SQLite 6
    5.1. Campos de la tabla peliculas 6
    5.2. Cómo se guarda un dato 7
6. Reglas de validación 7
    6.1. Películas 7
    6.2. Cuentas, perfiles y marcas 8
7. Autenticación y permisos con JWT 8
    7.1. Registro e inicio de sesión 8
    7.2. Rutas protegidas y roles 9
    7.3. Para qué sirve el token 9
    7.4. Medidas de seguridad añadidas 9
8. Rutas de la API 9
9. Manejo de errores 11
10. Personalización realizada 11
11. Interfaz web 11
12. Instalación y ejecución 14
13. Pruebas y evidencias 15

13.1. Alcance de la revisión actual 15
13.2. Pruebas a repetir en Postman 16
13.3. Evidencias de la versión actual 16
14. Cumplimiento de los requisitos 16
15. Conclusiones y mejoras 17
Anexo A. Capturas de pruebas 18

1. Introducción

Legión+ es una API REST desarrollada en Python con Flask para administrar un catálogo de películas. Permite listar y consultar registros de forma pública, mientras que crear, actualizar, eliminar y sincronizar películas requiere una cuenta de administrador. La información se almacena en SQLite y la interfaz web permite explorar la cartelera, consultar próximos estrenos y organizar preferencias mediante perfiles personales.

Esta edición actualiza la documentación a partir del código de Legion-Plus.zip. Describe el registro de cuentas, los roles, el panel de gestión, la configuración mediante variables de entorno y las nuevas medidas de seguridad. Las capturas del anexo provienen de una ejecución real de esta versión y se complementan con las comprobaciones locales realizadas sobre el código actualizado el 26 de octubre de 2026.

2. Objetivos

2.1. Objetivo general

Desarrollar y personalizar una API REST para gestionar el catálogo de Legión+ con Python, Flask y SQLite, utilizando autenticación JWT y permisos por rol para controlar las operaciones de administración, junto con una interfaz web que facilite la consulta de películas y el manejo de perfiles personales.

2.2. Objetivos específicos

- Organizar películas, cuentas, perfiles y preferencias en tablas relacionadas de SQLite.
- Implementar las cinco operaciones CRUD y restringir los cambios del catálogo a cuentas administradoras.
- Permitir el registro de usuarios y proteger sus perfiles mediante comprobaciones de pertenencia.
- Validar los datos y devolver mensajes claros con el código HTTP correspondiente.
- Ofrecer una interfaz pública y un panel de administración y documentar la instalación y las pruebas.

3. Descripción de la API

Cada película incluye título, director, género, año y calificación, además de información opcional como sinopsis, plataformas y enlaces al póster, al fondo y al tráiler. El catálogo distingue los estados disponible y proximamente. Los próximos estrenos requieren una fecha de lanzamiento y pueden registrarse sin calificación, que se guarda como 0. Los datos pueden ingresarse desde el panel o importarse mediante The Movie Database (TMDB).

La aplicación permite crear cuentas con el rol usuario. Estas cuentas administran sus propios perfiles y marcan películas como vista, viendo o no_interesa. Cada perfil pertenece a una cuenta

y puede tener una clave opcional de cuatro dígitos. El administrador se configura mediante variables de entorno y dispone de permisos adicionales. Registrarse desde la página no permite obtener ese rol.

La interfaz pública presenta el carrusel, el buscador, los filtros, las fichas y los próximos estrenos. El panel de administración se encuentra en una página independiente y permite crear, editar, eliminar y sincronizar películas. El servidor comprueba los permisos en cada solicitud protegida, incluso cuando se intenta acceder directamente a una ruta.

4. Tecnologías y estructura del proyecto

4.1. Tecnologías

Tabla 1. Tecnologías utilizadas

Tecnología	Uso
Python 3.10 o superior	Lenguaje del servidor.
Flask	Aplicación, blueprints, rutas y respuestas HTTP.
SQLite / sqlite3	Persistencia en un archivo local.
PyJWT	Generación y validación de tokens con HS256.
Werkzeug	Generación y comprobación de hashes; dependencia de Flask.
HTML, CSS y JavaScript	Interfaz pública y panel sin un framework de interfaz.
Postman	Peticiones y evidencias manuales anteriores.
TMDB, PokeAPI y Rick and Morty	Servicios externos para películas y avatares.

requirements.txt declara Flask y PyJWT sin fijar versiones exactas. La versión instalada depende del entorno. Las versiones visibles en capturas históricas no se presentan como versiones verificadas de esta entrega.

4.2. Archivos del proyecto

Tabla 2. Archivos y carpetas principales

Archivo o carpeta	Función
app.py	Crea Flask, registra blueprints, inicializa la base y añade cabeceras de seguridad.
config.py / .env.example	Carga variables y ofrece una plantilla de configuración.
database.py	Crea las cuatro tablas y habilita claves foráneas.
auth.py	Registro, login, sesión y decoradores token_requerido y solo_admin.
limitador.py	Cuenta intentos fallidos y aplica bloqueo temporal.

Archivo o carpeta	Función
peliculas.py / repositorio.py	Rutas de películas y operaciones SQL.
perfiles.py / avatares.py	Perfiles por cuenta, marcas y galería.
validaciones.py / errores.py	Validaciones y respuestas de error.
tmdb.py	Importación y actualización desde TMDB.
static/index.html	Estructura de la interfaz pública.
static/admin.html	Estructura del panel de administración.
static/css/estilos.css	Diseño visual y adaptación de pantalla.
static/js/api.js	Peticiones HTTP, token, rol y comunicación con la API.
static/js/app.js / ui.js	Eventos y estado de la página pública; elementos visuales.
static/js/admin.js	Formularios, tabla y acciones del panel.
templates/error.html	Errores presentados como página HTML.
verificar.py	Comprobaciones HTTP con el servidor encendido.
requirements.txt	Dependencias directas: Flask y PyJWT.

5. Base de datos SQLite

El archivo se llama peliculas.db por defecto y puede cambiarse con DB_NAME. Se crea al iniciar el servidor cuando no existe. init_db() utiliza CREATE TABLE IF NOT EXISTS y agrega determinadas columnas que falten en bases anteriores. La información permanece en disco después de reiniciar y el arranque no borra registros. La distribución revisada no incluye un archivo .db.

Tabla 3. Tablas y relaciones

Tabla	Contenido y relación
peliculas	Catálogo y datos de cada película.
usuarios	Nombre único, hash de contraseña y rol.
perfiles	Nombre, avatar, hash opcional de clave y usuario_id del propietario.
vistas	Relaciona perfil_id y pelicula_id con una marca. La combinación de ambos identificadores es única.

5.1. Campos de la tabla peliculas

Tabla 4. Campos del catálogo

Campo	Tipo SQLite	Condición o uso
id	INTEGER PK AUTOINCREMENT	Identificador automático.

Campo	Tipo SQLite	Condición o uso
titulo	TEXT NOT NULL	Obligatorio.
director	TEXT NOT NULL	Obligatorio.
genero	TEXT NOT NULL	Obligatorio y dentro de la lista.
anio	INTEGER NOT NULL	Obligatorio y dentro del rango.
calificacion	REAL NOT NULL	Obligatoria para disponibles; 0 si se omite en próximos estrenos.
sinopsis	TEXT	Opcional.
poster_url	TEXT	Opcional; prefijo http:// o https://.
estado	TEXT NOT NULL	Predeterminado: disponible.
fecha_estreno	TEXT	Exigida por la API para proximamente.
plataformas	TEXT	Opcional.
trailer_url	TEXT	Opcional; dominio de YouTube.
fondo_url	TEXT	Opcional; prefijo http:// o https://.

usuarios contiene id, username, clave y rol. username es único y el rol predeterminado es usuario. perfiles contiene id, usuario_id, nombre, avatar y clave. Las respuestas omiten el hash del perfil e incluyen tiene_clave. vistas contiene id, perfil_id, pelicula_id y estado. Las conexiones activan PRAGMA foreign_keys = ON; antes de eliminar una película o un perfil se borran sus marcas relacionadas dentro de la misma transacción.

5.2. Cómo se guarda un dato

El cliente envía POST /peliculas con JSON y Authorization: Bearer <token>. Primero se comprueba la firma y el vencimiento del token y después se exige el rol admin. cuerpo_valido y validar_pelicula revisan la estructura y las reglas. Una solicitud sin token válido devuelve 401 y una cuenta sin permisos recibe 403 antes de ejecutar la operación.

repositorio.crear_pelicula ejecuta un INSERT parametrizado con signos de interrogación para los valores. conn.commit() confirma la transacción y SQLite asigna el identificador. La API consulta el registro creado y devuelve su contenido con 201 Created. AUTOINCREMENT evita reutilizar identificadores previamente confirmados, pero no garantiza una secuencia sin saltos ni que el siguiente número sea siempre el del registro borrado más uno.

6. Reglas de validación

6.1. Películas

- El cuerpo debe ser un objeto JSON. titulo, director, genero y anio son obligatorios; calificacion también, salvo para proximamente.

- titulo y director deben ser textos no vacíos de hasta 200 caracteres. anio debe ser entero y no admite booleanos.
- El año permitido va desde 1888 hasta el año actual; para próximos estrenos se admiten hasta diez años adicionales. La calificación debe ser numérica y estar entre 0 y 10.
- Los géneros son Acción, Animación, Ciencia ficción, Comedia, Drama, Romance y Terror. El estado admite disponible o proximamente.
- fecha_estreno es obligatoria para proximamente. Se comprueba su presencia y tipo de texto, pero no una fecha de calendario completa.
- Los opcionales deben ser texto o null. La sinopsis admite hasta 5.000 caracteres; enlaces y plataformas hasta 500; fecha_estreno hasta 50.
- poster_url y fondo_url deben comenzar por http:// o https:// cuando tienen contenido. El host del tráiler debe ser youtube.com, youtu.be o un subdominio de estos.

Las reglas se aplican al crear y al actualizar. Se devuelve el primer error con 400 y un mensaje que identifica el problema. Por ejemplo: El campo "calificacion" debe estar entre 0 y 10.

6.2. Cuentas, perfiles y marcas

- username debe ser texto no vacío de hasta 30 caracteres y admite letras ASCII, números, punto, guion y guion bajo. La contraseña de registro debe tener entre 8 y 200 caracteres.
- El registro crea siempre el rol usuario. Un nombre que ya existe produce 409.
- El nombre del perfil admite hasta 20 caracteres y el avatar debe existir en la galería. La clave es opcional; si se envía debe ser un texto de cuatro dígitos.
- Las marcas admiten vista, viendo y no_interesa. Las rutas del perfil comprueban que pertenezca a la cuenta autenticada.

7. Autenticación y permisos con JWT

7.1. Registro e inicio de sesión

POST /registro crea una cuenta normal y responde 201 con un token y los datos públicos del usuario. POST /login compara la contraseña con el hash almacenado y responde 200 con token y usuario, que incluye id, username y rol. El administrador se crea o actualiza al iniciar a partir de ADMIN_USER y ADMIN_PASSWORD o ADMIN_PASSWORD_HASH. Ya no se utilizan las credenciales fijas documentadas en la versión anterior.

```
POST /login
Content-Type: application/json

{"username": "TU_USUARIO", "password": "TU_CONTRASENA"}

200 OK
{"token": "<JWT>",
 "usuario": {"id": 1, "username": "TU_USUARIO", "rol": "admin"}}
```

7.2. Rutas protegidas y roles

El token se firma con HS256 y SECRET_KEY e incluye id, username, rol y exp. Dura 60 minutos por defecto y se configura con MINUTOS_EXPIRACION_TOKEN. token_requerido valida la cabecera Bearer, la firma y el vencimiento. solo_admin incorpora esa validación y exige el rol admin para crear, actualizar, eliminar películas y sincronizar con TMDB. Las consultas del catálogo son públicas y GET /sesion permite consultar la cuenta del token.

Las rutas de perfiles y marcas exigen autenticación y comprueban la propiedad del perfil. Intentar acceder al perfil de otra cuenta devuelve 404. Esta verificación es independiente de la clave opcional del perfil, cuya comprobación se realiza en la ruta /entrar.

7.3. Para qué sirve el token

El token acredita el inicio de sesión sin enviar la contraseña en cada operación. El servidor verifica la firma y rechaza tokens alterados o vencidos con 401. La firma comprueba autenticidad e integridad, pero no cifra el contenido. El navegador guarda token y rol en localStorage; el rol guardado sirve para mostrar opciones, mientras que el servidor decide si la operación está autorizada.

No se mantiene una sesión por usuario en el servidor ni se incorpora una lista de revocación. El rol se lee del token emitido. Si no se define SECRET_KEY se genera una clave temporal y los tokens dejan de ser válidos al reiniciar. Debe configurarse una clave persistente y reservada para mantener un comportamiento estable.

7.4. Medidas de seguridad añadidas

- Contraseñas de cuentas y claves de perfiles guardadas como hashes con sal de Werkzeug. El algoritmo predeterminado de generación en las versiones actuales es scrypt.
- Migración de claves antiguas de perfil basadas en SHA-256 si se configura SAL_LEGADA y se verifica la clave anterior.
- Bloqueo tras cinco fallos por defecto durante una ventana de 300 segundos. El contador de login se basa en IP y el de perfil en perfil e IP. Devuelve 429. Los contadores viven en memoria y se reinician con el proceso.
- Límite del cuerpo de petición de 1 MB y límites de longitud en los campos.
- Cabeceras Content-Security-Policy, X-Content-Type-Options, X-Frame-Options y Referrer-Policy. Debug desactivado por defecto.

8. Rutas de la API

La URL base local es http://127.0.0.1:5000. Las llamadas devuelven JSON; los errores pueden mostrarse en HTML si Accept da preferencia a ese formato. Las rutas protegidas reciben Authorization: Bearer <token>. La expresión <id> representa un identificador entero.

Tabla 5. Rutas de cuentas y películas

Método y ruta	Acceso	Éxito y función
GET /	Público	200. Bienvenida.
POST /registro	Público	201. Cuenta normal y token.
POST /login	Público	200. Token y usuario.
GET /sesion	JWT	200. Datos de sesión.
GET /peliculas	Público	200. Listado por id.
GET /peliculas/<id>	Público	200. Consulta individual.
POST /peliculas	Admin	201. Crear película.
PUT /peliculas/<id>	Admin	200. Actualizar película.
DELETE /peliculas/<id>	Admin	200. Eliminar película.
POST /sincronizar	Admin	200. Importar desde TMDB.

Tabla 6. Rutas de perfiles y preferencias

Método y ruta	Acceso	Función
GET /avatares	Público	Consultar galería.
GET /perfiles	JWT	Listar perfiles propios.
POST /perfiles	JWT	Crear perfil propio (201).
PUT /perfiles/<id>	Propietario	Actualizar perfil.
DELETE /perfiles/<id>	Propietario	Eliminar perfil y marcas.
POST /perfiles/<id>/entrar	Propietario	Comprobar clave opcional.
GET /perfiles/<id>/vistas	Propietario	Consultar marcas.
PUT /perfiles/<id>/vistas/<pelicula>	Propietario	Crear o actualizar marca.
DELETE /perfiles/<id>/vistas/<pelicula>	Propietario	Quitar marca.

PUT reemplaza los campos del registro y exige reenviar los obligatorios. Los opcionales omitidos pasan a null y estado vuelve a disponible si se omite o es null. Un id no entero no coincide con la ruta. Autenticación y rol se comprueban antes de buscar la película en las operaciones administrativas.

```
{
  "titulo": "Película de prueba",
  "director": "Dirección de prueba",
  "genero": "Drama",
  "anio": 2024,
  "calificacion": 8.5,
  "estado": "disponible"
}
```

9. Manejo de errores

errores.py centraliza respuestas con la estructura {"error": "mensaje"}. Si el cliente prefiere HTML se utiliza templates/error.html. Los códigos comunican qué ocurrió y permiten diferenciar problemas de datos, autenticación, permisos o recursos.

Tabla 7. Códigos HTTP de error

Código	Situación
400 Bad Request	JSON ausente, datos inválidos o falta de clave TMDB.
401 Unauthorized	Credenciales incorrectas, token ausente, inválido o vencido; clave de perfil incorrecta.
403 Forbidden	El rol no permite administrar películas o sincronizar.
404 Not Found	Película, perfil o ruta inexistente; también perfil de otra cuenta.
405 Method Not Allowed	Método no admitido.
409 Conflict	Nombre de usuario ya registrado.
413 Content Too Large	Cuerpo superior a 1 MB.
429 Too Many Requests	Demasiados intentos fallidos.
500 Internal Server Error	Fallo inesperado.
502 Bad Gateway	Error al conectar con TMDB.

10. Personalización realizada

El proyecto adapta tablas, campos, consultas SQL, rutas, mensajes y reglas al catálogo de Legión+. Esta versión amplía la personalización con cuentas y roles, perfiles por propietario, listas personales, registro y panel de administración. Integra TMDB para completar el catálogo y utiliza avatares locales y servicios externos para personalizar los perfiles.

La configuración deja de depender de usuarios y claves escritos en el código. Los valores privados se leen del entorno o de .env y se entrega .env.example como guía. La base se inicializa con cuatro tablas y el administrador se configura durante el arranque.

11. Interfaz web

La página pública permite explorar películas mediante carrusel, búsqueda por título o director, filtros por género y orden de resultados. Muestra próximos estrenos y fichas con información, plataformas y tráiler. Al crear una cuenta o iniciar sesión se habilitan perfiles propios y marcas. El mantenimiento de películas se realiza desde el panel independiente de administración.

CSS define colores, tamaños y distribución. api.js comunica el navegador con el servidor y maneja token y rol. app.js coordina la página pública y ui.js construye sus elementos visuales.

index.html define su estructura. admin.html y admin.js contienen la estructura y el funcionamiento del panel para crear, editar, eliminar y sincronizar.

Las siguientes capturas provienen de docs/capturas del ZIP actualizado. Son imágenes suministradas con el proyecto, no evidencia de una nueva ejecución del navegador durante esta revisión.

Figura 1. Cartelera pública.

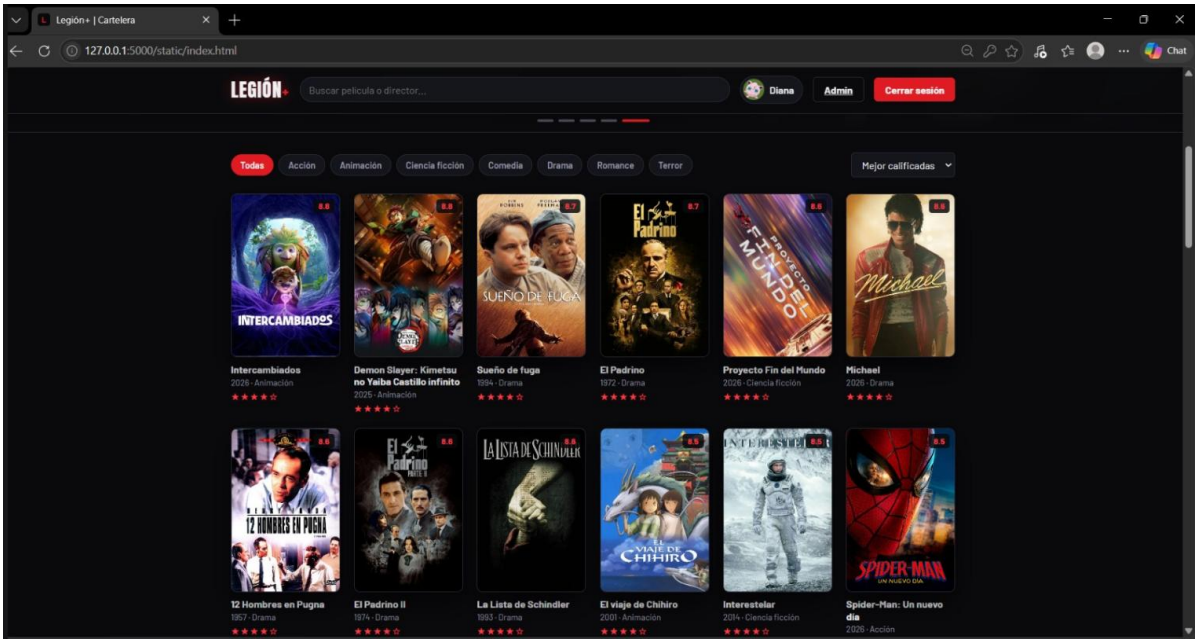


Figura 2. Ficha de una película.

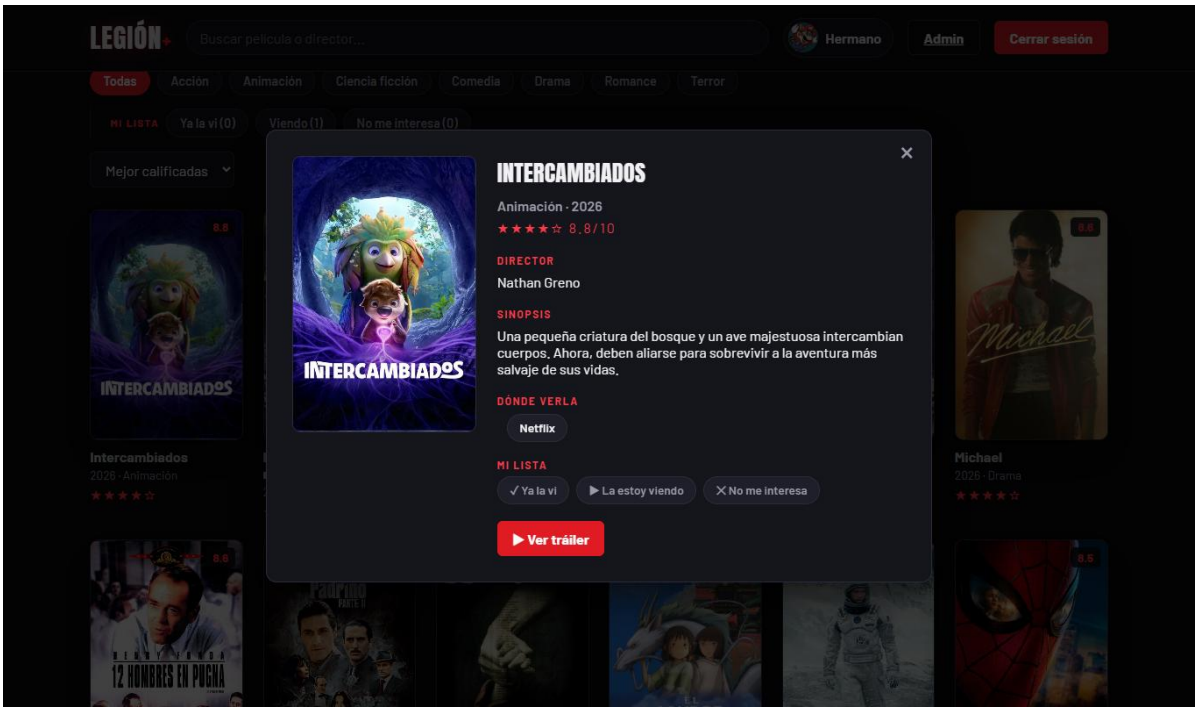
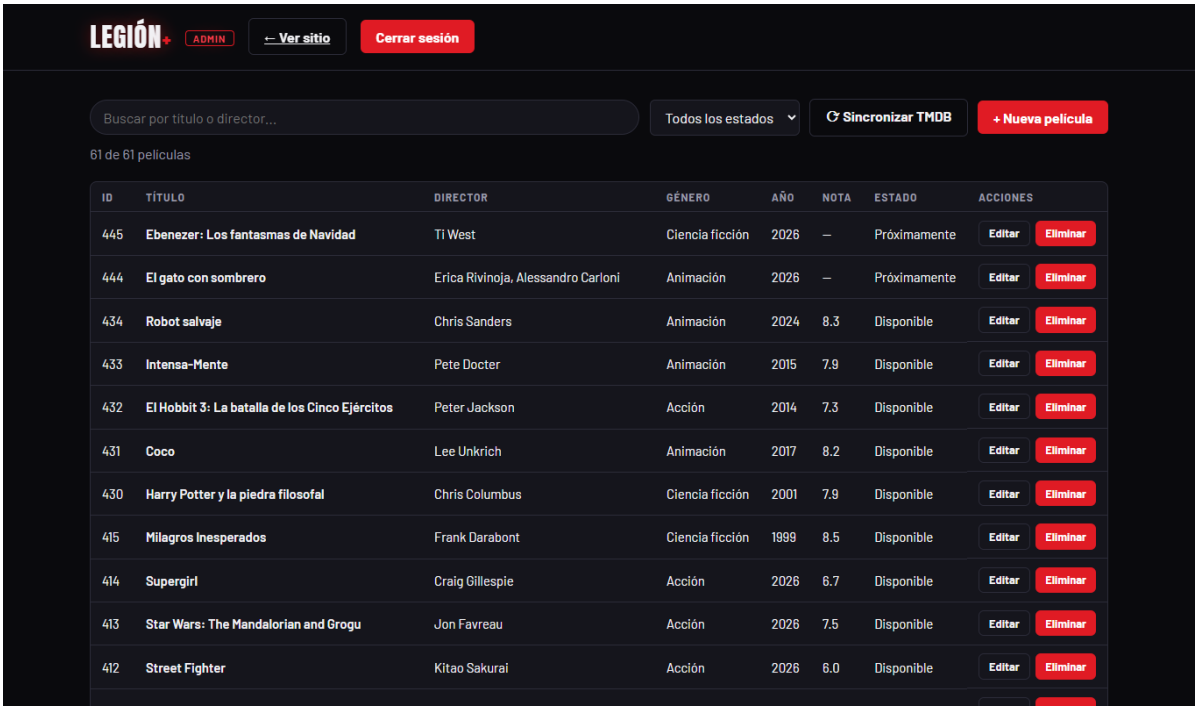


Figura 3. Panel de administración de películas.



12. Instalación y ejecución

Se necesita Python 3.10 o superior. Descomprima el proyecto y abra en Visual Studio Code la carpeta con app.py, requirements.txt y .env.example. Ejecute los comandos desde esa carpeta. El entorno virtual se crea en cada equipo y no debe incluirse en la entrega.

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
Copy-Item .env.example .env
```

Abra .env y complete ADMIN_USER y ADMIN_PASSWORD. Puede definir ADMIN_PASSWORD_HASH en lugar de la contraseña. SECRET_KEY debe mantenerse estable y reservada. Para generar una clave aleatoria ejecute el comando siguiente y copie el resultado al archivo .env:

```
.\.venv\Scripts\python.exe -c "import secrets; print(secrets.token_hex(32))"
```

Tabla 8. Variables de configuración

Variable	Uso
ADMIN_USER	Nombre del administrador; obligatorio.
ADMIN_PASSWORD / ADMIN_PASSWORD_HASH	Contraseña o hash; se requiere uno. verificar.py utiliza ADMIN_PASSWORD.
SECRET_KEY	Firma JWT. Si falta se crea una temporal.
TMDB_API_KEY	Necesaria para importar; no para arrancar ni crear películas manualmente.
DB_NAME	Archivo SQLite; por defecto peliculas.db.
MINUTOS_EXPIRACION_TOKEN	Por defecto 60 minutos.
MAX_INTENTOS / SEGUNDOS_BLOQUEO	Valores predeterminados: 5 y 300.
SAL_LEGADA	Solo para migrar claves antiguas de perfil.
FLASK_DEBUG	0 por defecto; 1 habilita depuración.

Guarde la configuración e inicie el servidor:

```
.\.venv\Scripts\python.exe app.py
```

- API: http://127.0.0.1:5000
- Interfaz: http://127.0.0.1:5000/static/index.html
- Administración: http://127.0.0.1:5000/static/admin.html

La base se crea al arrancar con la cuenta administradora configurada, pero el catálogo comienza vacío si no existe una base anterior. Puede llenarse manualmente desde el panel o mediante TMDB con clave válida y cuenta admin. La sincronización no se ejecuta automáticamente al iniciar. Se detiene el servidor con Ctrl+C y se mantiene .env fuera de la entrega pública.

Para sincronizar en Postman obtenga el token administrador con POST /login y envíe POST /sincronizar con Bearer Token y cuerpo {}. Puede enviar {"buscar": ["Interstellar"]}; se admiten hasta 20 títulos de texto. La integración consulta las películas por título para decidir si inserta o actualiza.

13. Pruebas y evidencias

13.1. Alcance de la revisión actual

El 26 de octubre de 2026 se revisó el ZIP actualizado y se ejecutaron 23 comprobaciones locales con el cliente de pruebas de Flask y SQLite temporal. Todas devolvieron los resultados esperados. Se utilizaron cuentas de prueba y una galería simulada para evitar llamadas externas. No se modificó la base del usuario. Esto no sustituye una prueba completa del navegador, de TMDB ni de producción.

Tabla 9. Comprobaciones locales de la versión actual

N.º	Comprobación	Resultado
1	Inicio de sesión administrador	200 / correcto
2	Registro y rol de usuario	201 / correcto
3	Registro duplicado	409 / correcto
4	Consulta de sesión	200 / correcto
5	Crear sin token	401 / correcto
6	Crear con rol usuario	403 / correcto
7	Crear película como administrador	201 / correcto
8	Listar películas sin sesión	200 / correcto
9	Consultar película sin sesión	200 / correcto
10	Actualizar película	200 / correcto
11	Validación de calificación	400 / correcto
12	Validación de dominio del tráiler	400 / correcto
13	Crear perfil propio	201 / correcto
14	Aislamiento entre cuentas	404 / correcto
15	Marcar película en perfil propio	200 / correcto
16	Eliminar película y sus marcas	200 / correcto
17	Consultar película eliminada	404 / correcto
18	Método no admitido	405 / correcto
19	Sincronización sin permisos	403 / correcto
20	Sincronización sin clave TMDB	400 / correcto
21	Límite de tamaño del cuerpo	413 / correcto

N.º	Comprobación	Resultado
22	Límite de intentos fallidos	429 / correcto
23	Cabeceras de seguridad presentes	200 / correcto

La comprobación de cabeceras verifica su presencia, no toda la política CSP. La respuesta 400 de sincronización verifica la ausencia de clave, no una importación real. No se ejecutó Newman durante esta revisión.

13.2. Pruebas a repetir en Postman

- Iniciar sesión como administrador y comprobar token y usuario con rol admin.
- Crear dos películas, listar y consultar sus identificadores, actualizar una y eliminar la otra.
- Probar una operación sin token para obtener 401 y con cuenta normal para obtener 403.
- Enviar calificación fuera de rango para obtener 400 y consultar la película eliminada para obtener 404.
- Registrar una cuenta y repetir el nombre para obtener 409. Comprobar el aislamiento de perfiles.
- Reiniciar el servidor y confirmar persistencia; con SECRET_KEY estable comprobar también la vigencia del token.
- Probar sincronización real, panel, registro y perfiles desde el navegador.

Con el servidor encendido puede ejecutarse verificar.py en otra terminal. El código contiene 57 comprobaciones sobre autenticación, operaciones CRUD, validaciones, errores, perfiles, roles y catálogo; el README indica la misma cifra. El script utiliza la configuración del administrador y requiere un catálogo previamente cargado para sus últimas comprobaciones. Su resultado completo debe confirmarse al ejecutarlo; las 23 comprobaciones adicionales de esta revisión se presentan por separado.

```
.\.venv\Scripts\python.exe verificar.py
```

13.3. Evidencias de la versión actual

El Anexo A reúne 25 capturas de una ejecución real de la API sobre una base de datos vacía, con cuentas de demostración. Cada una muestra la petición, la respuesta y el código HTTP. Cubren registro, inicio de sesión, las cinco operaciones CRUD con dos registros, los errores 400, 401, 403, 404, 405 y 409, los roles y el aislamiento entre cuentas.

14. Cumplimiento de los requisitos

Tabla 10. Correspondencia entre requisitos y desarrollo

Requisito	Dónde se documenta
Temática y finalidad	Secciones 1 a 3.

Requisito	Dónde se documenta
SQLite, id automático y campos propios	Sección 5.
Cinco operaciones CRUD	Secciones 5.2, 8 y 13.
JWT para modificar datos	Sección 7; además se exige rol admin.
Obligatorios, tipos y reglas propias	Sección 6.
Errores 400, 401 y 404	Secciones 9 y 13.
Personalización coherente	Secciones 4 y 10.
Dos registros, cambios y persistencia	Anexo A, figuras 9 a 16; persistencia en 13.2.
Instalación y entrega sin .venv	Sección 12.
Campos, reglas y rutas	Secciones 5, 6 y 8.
Evidencias de login, CRUD y errores	Sección 13 y anexo.
Explicar guardado y token	Secciones 5.2 y 7.3.

15. Conclusiones y mejoras

La versión actual mantiene las cinco operaciones y amplía el control de acceso con cuentas, roles y perfiles por propietario. Las operaciones administrativas exigen JWT válido con rol admin. Se añaden configuración en .env, hashes de contraseñas, límites de intentos y cabeceras de seguridad. Las 23 comprobaciones locales fueron satisfactorias dentro del alcance indicado.

Antes de publicar conviene probar el navegador y los servicios externos, usar HTTPS y un servidor WSGI de producción y mantener las claves fuera del repositorio. Se puede mejorar la validación de fechas y enlaces, incorporar revocación o renovación de tokens y compartir los contadores de intentos entre procesos. La clave opcional de perfil podría reforzarse con autorización temporal exigida en las acciones posteriores al desbloqueo.

Mover secretos al entorno, almacenar contraseñas con hash y desactivar debug por defecto ya son funciones implementadas y dejan de figurar como mejoras pendientes.

Anexo A. Capturas de pruebas

En las siguientes imágenes mostramos las pruebas de registro, inicio de sesión, creación, consulta, actualización y eliminación de películas. También presentamos las respuestas de error y las pruebas de roles y aislamiento entre cuentas. Las peticiones se ejecutaron sobre una base de datos vacía con cuentas de demostración, y cada captura muestra la petición enviada, la respuesta del servidor y su código HTTP.

Figura 4. Registrar una cuenta de usuario (201).

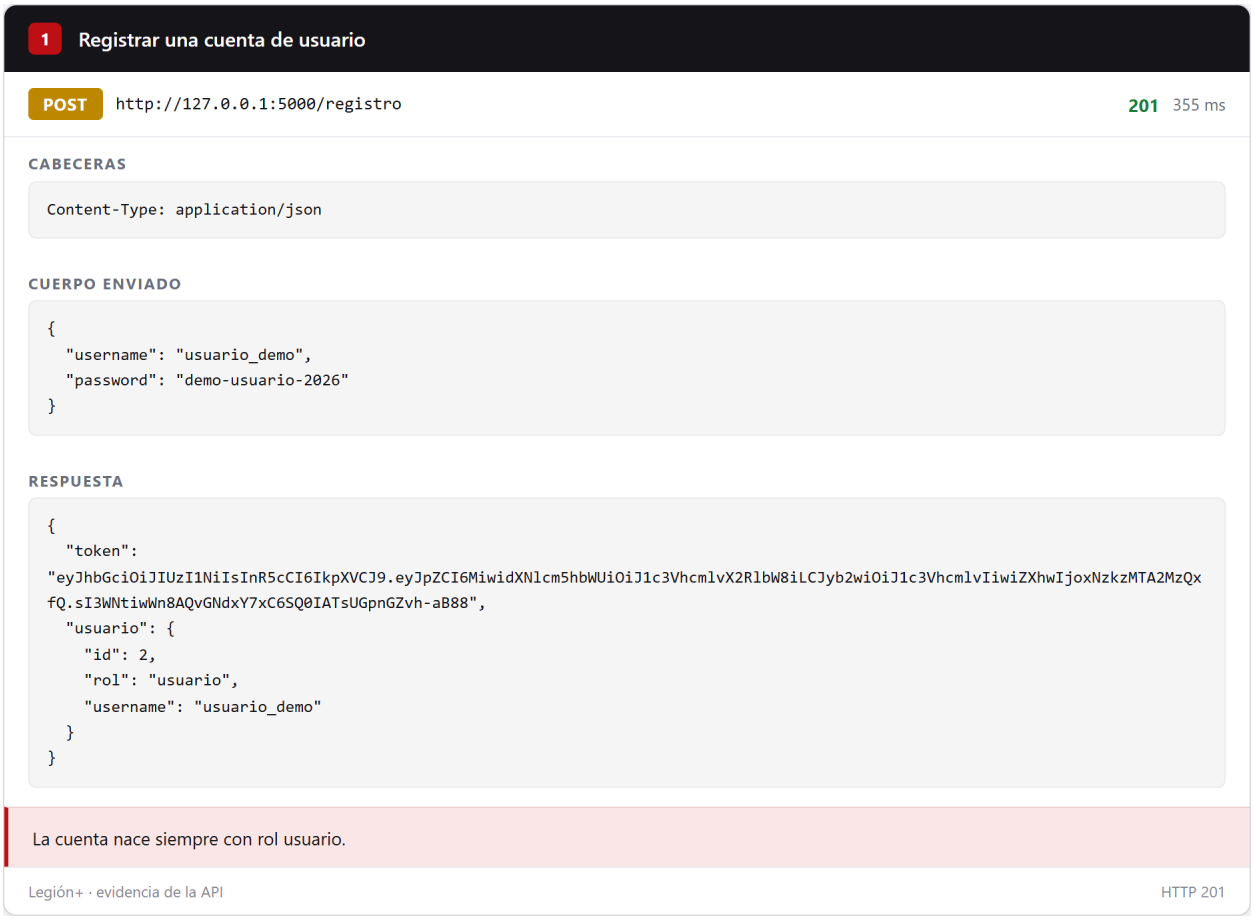


Figura 5. Registrar el mismo nombre otra vez (409).

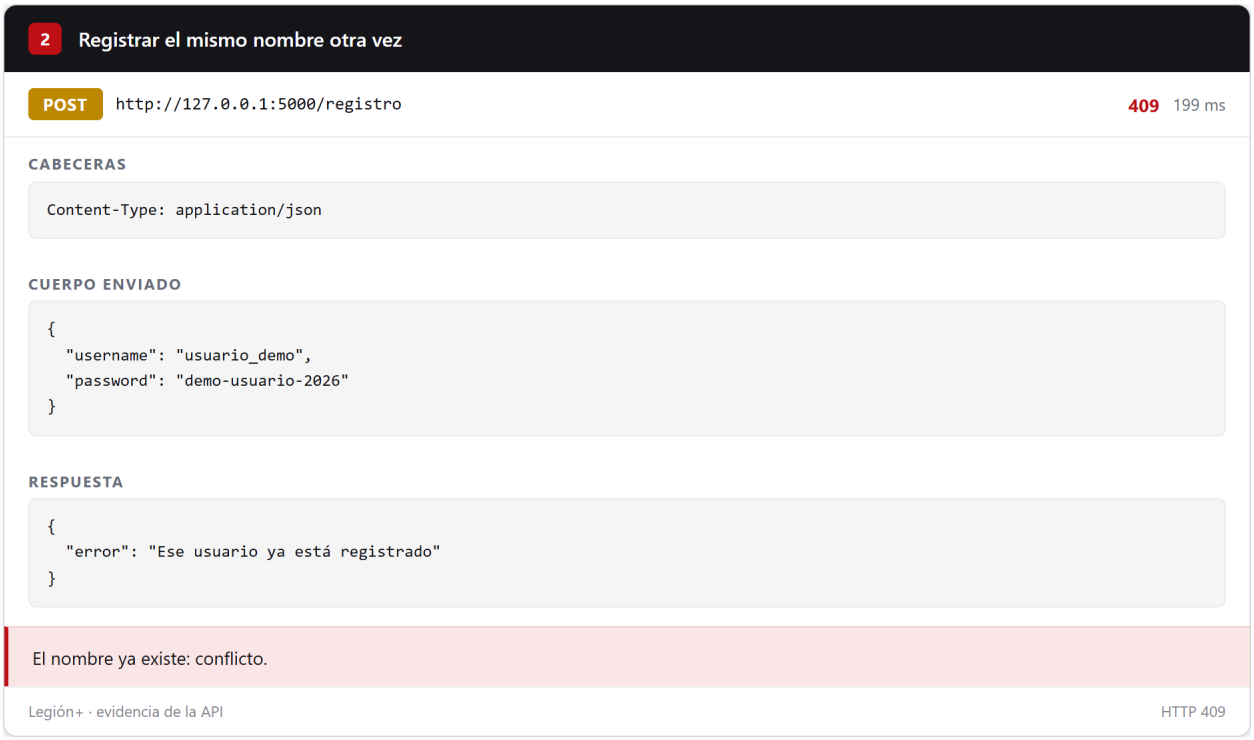


Figura 6. Iniciar sesión como administrador (200).

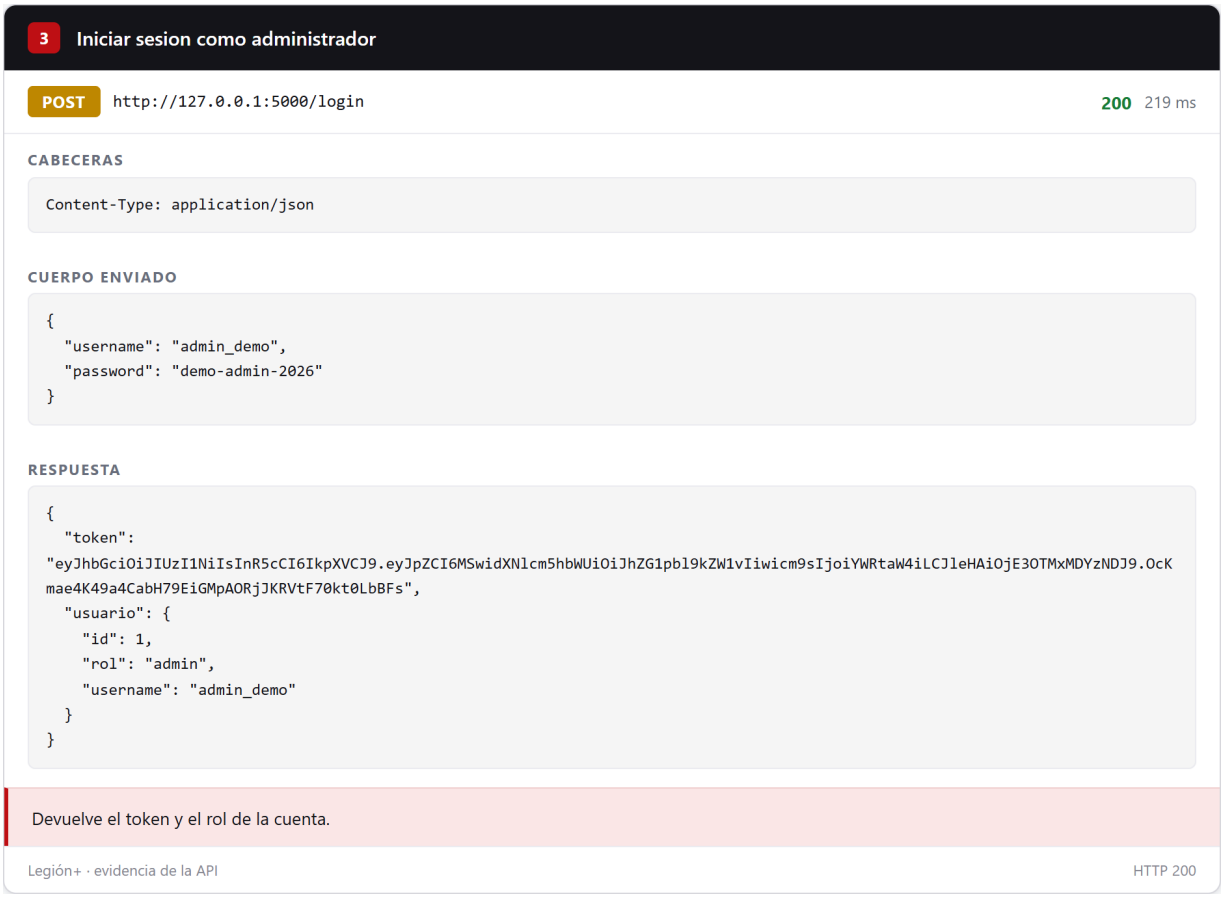


Figura 7. Contraseña incorrecta (401).

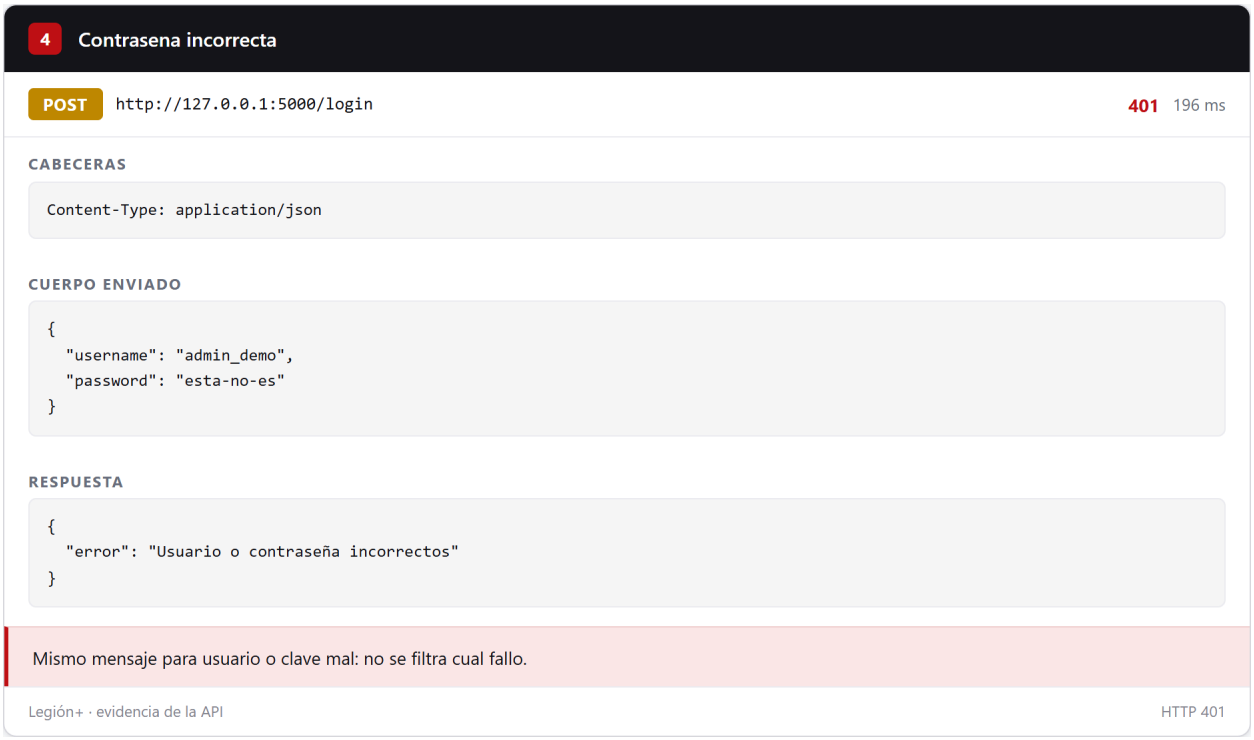


Figura 8. Consultar la sesión del token (200).

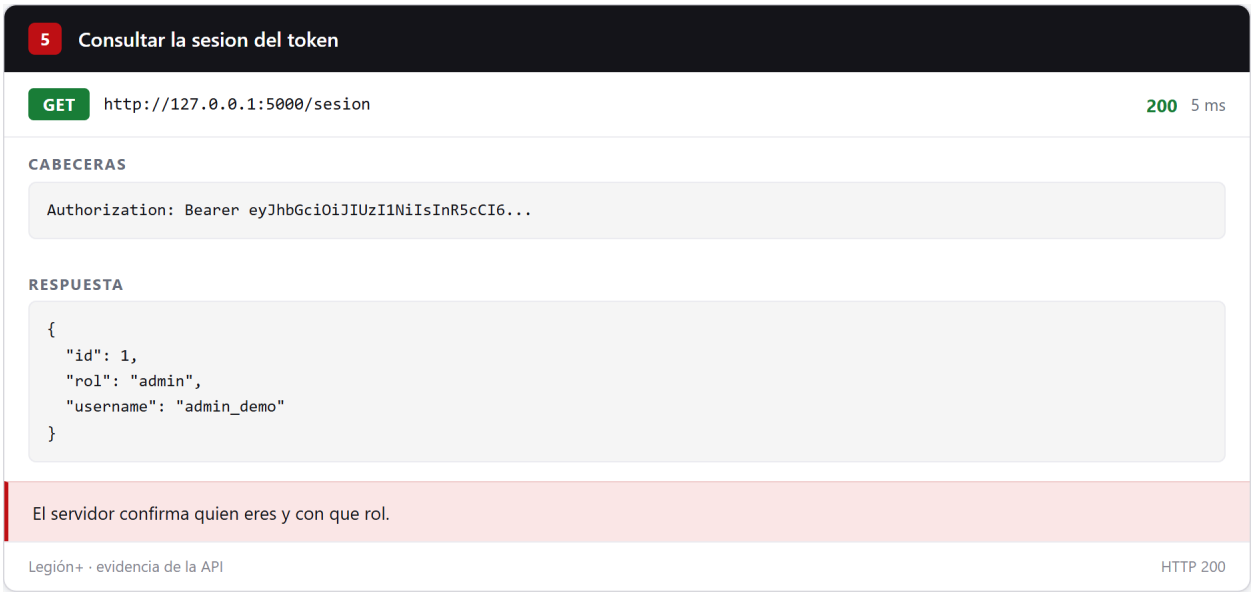


Figura 9. Crear una película (201).

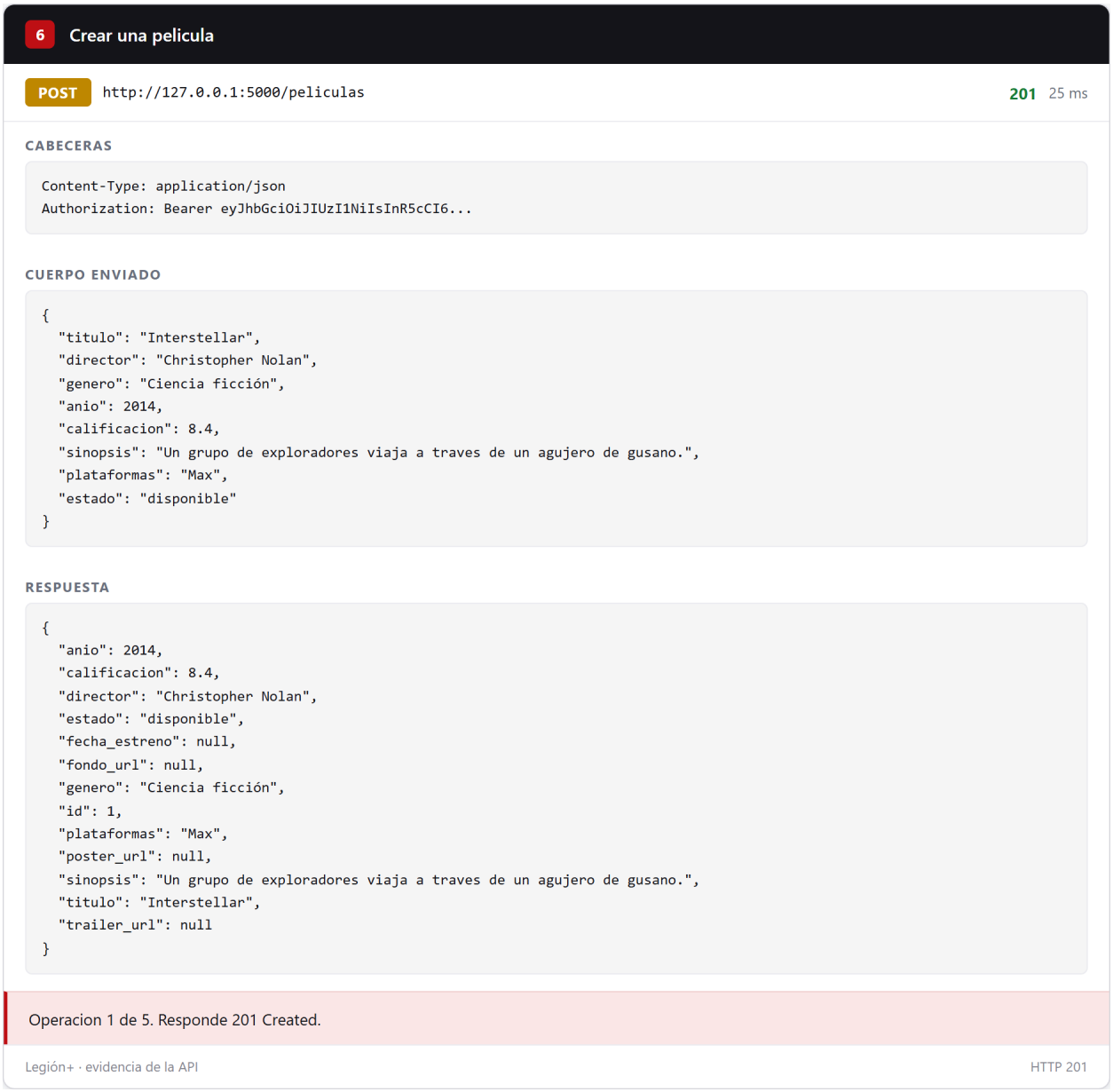


Figura 10. Crear un próximo estreno (201).

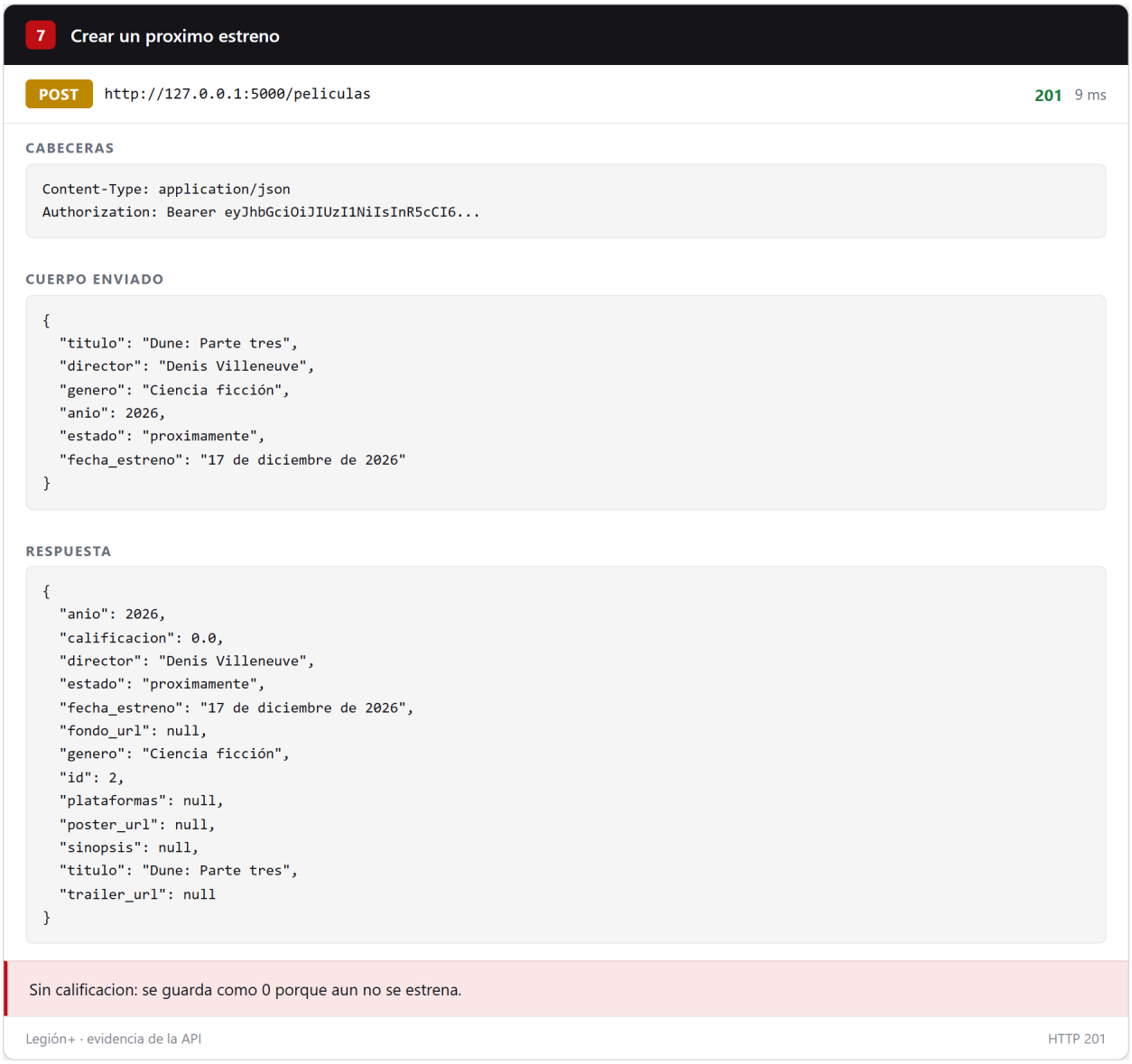


Figura 11. Listar todas las películas (200).

8 Listar todas las peliculas

GET http://127.0.0.1:5000/peliculas 200 4 ms

RESPUESTA


```
[
  {
    "anio": 2014,
    "calificacion": 8.4,
    "director": "Christopher Nolan",
    "estado": "disponible",
    "fecha_estreno": null,
    "fondo_url": null,
    "genero": "Ciencia ficción",
    "id": 1,
    "plataformas": "Max",
    "poster_url": null,
    "sinopsis": "Un grupo de exploradores viaja a traves de un agujero de gusano.",
    "titulo": "Interstellar",
    "trailer_url": null
  },
```


Operacion 2 de 5. Publica: no necesita token.

Legión+ · evidencia de la API HTTP 200

Figura 12. Consultar una película por id (200).

9 Consultar una pelicula por id

GET http://127.0.0.1:5000/peliculas/1 200 5 ms

RESPUESTA


```
{
  "anio": 2014,
  "calificacion": 8.4,
  "director": "Christopher Nolan",
  "estado": "disponible",
  "fecha_estreno": null,
  "fondo_url": null,
  "genero": "Ciencia ficción",
  "id": 1,
  "plataformas": "Max",
  "poster_url": null,
  "sinopsis": "Un grupo de exploradores viaja a traves de un agujero de gusano.",
  "titulo": "Interstellar",
  "trailer_url": null
}
```


Operacion 3 de 5. Tambien publica.

Legión+ · evidencia de la API HTTP 200

Figura 13. Actualizar una película (200).

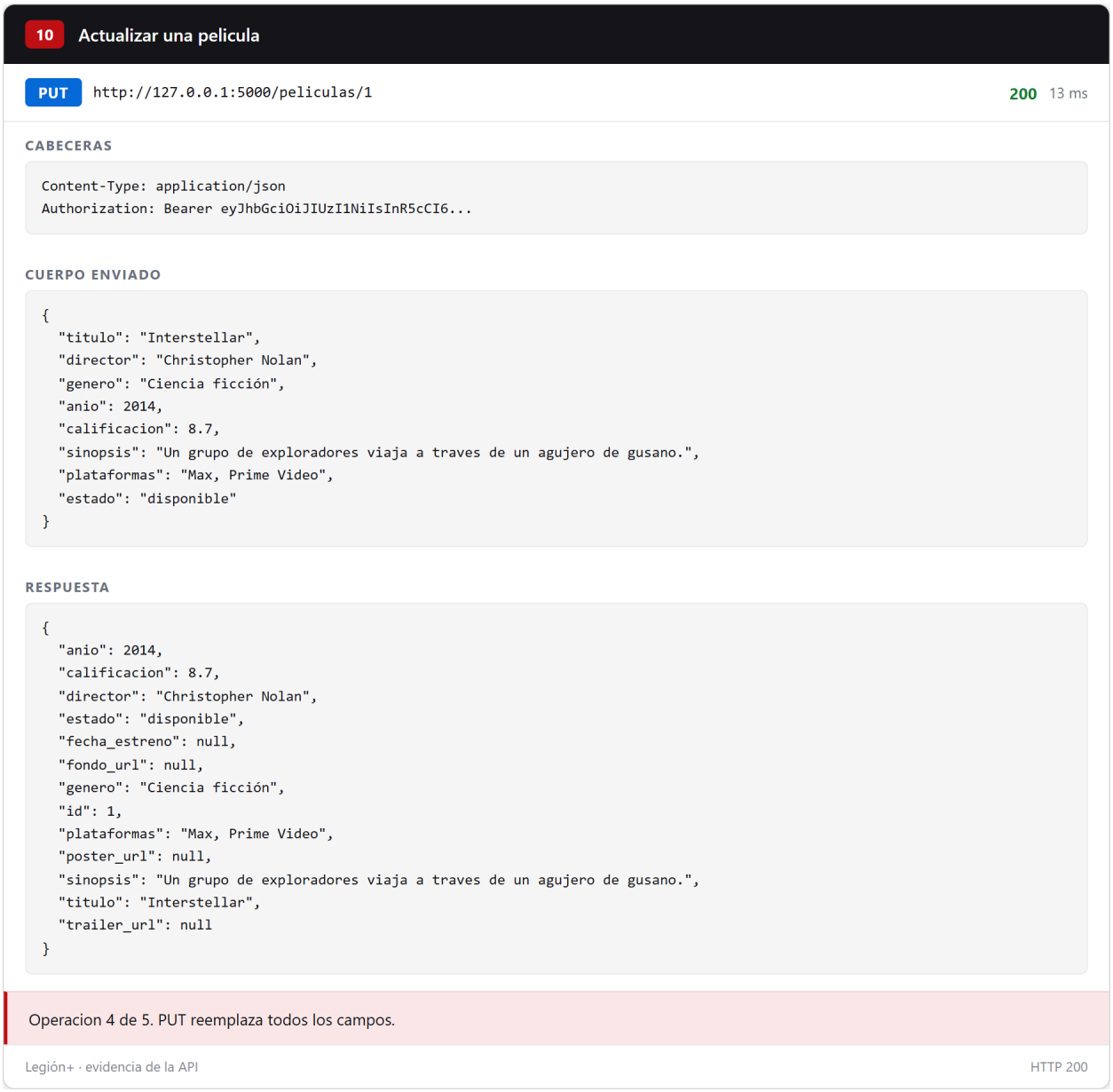


Figura 14. Eliminar una película (200).



Figura 15. La película eliminada ya no existe (404).

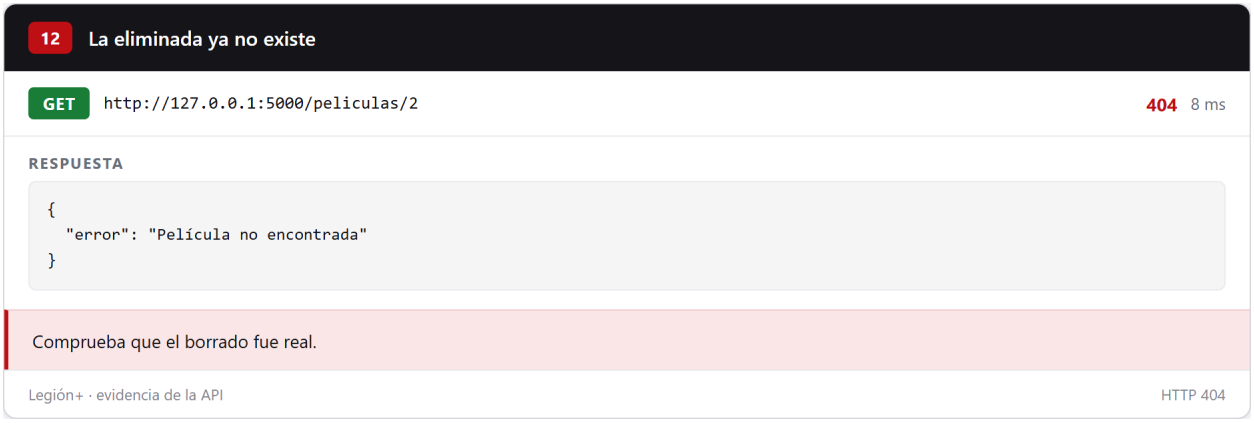


Figura 16. La otra película sigue guardada y con el cambio (200).



Figura 17. Error 400 por datos inválidos.



Figura 18. Error 401 en ruta protegida sin token.

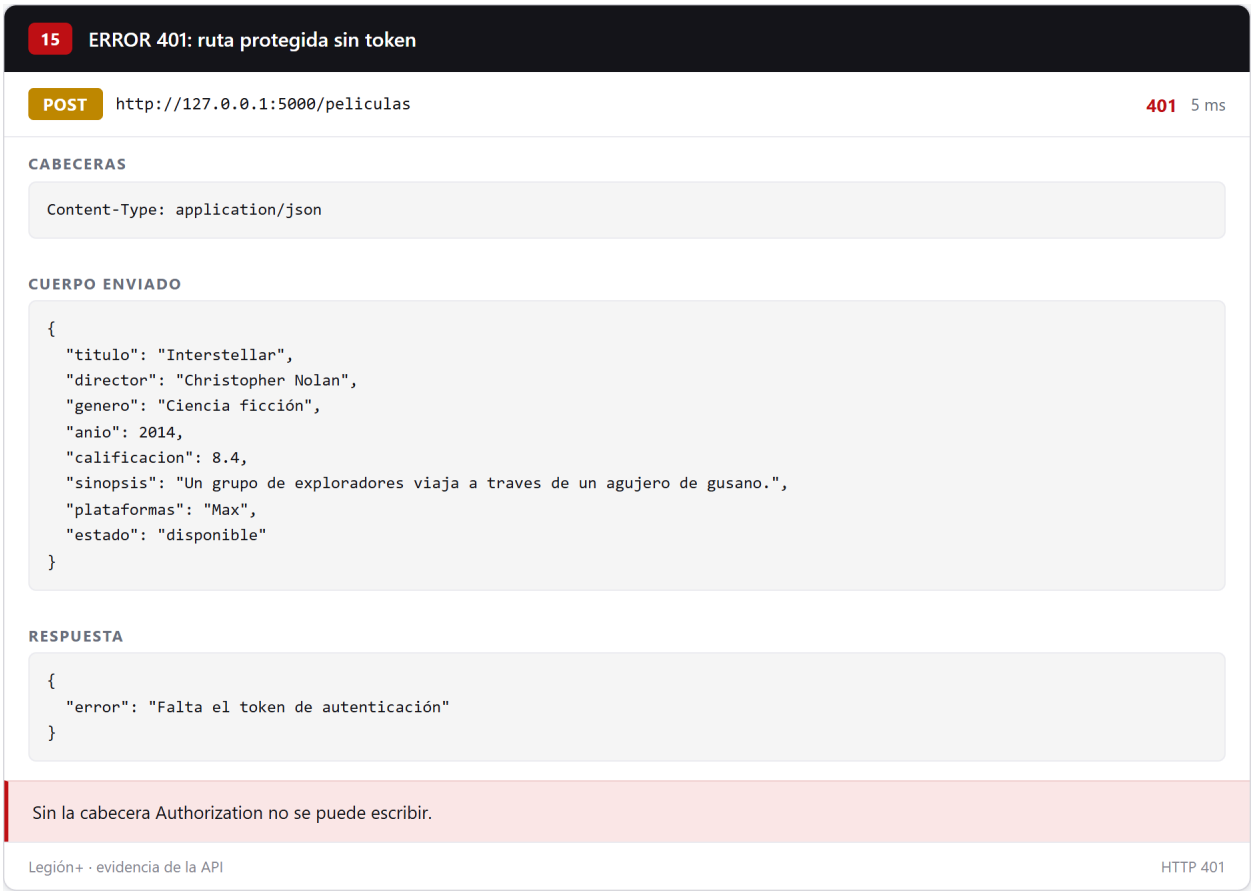


Figura 19. Error 404 por id inexistente.

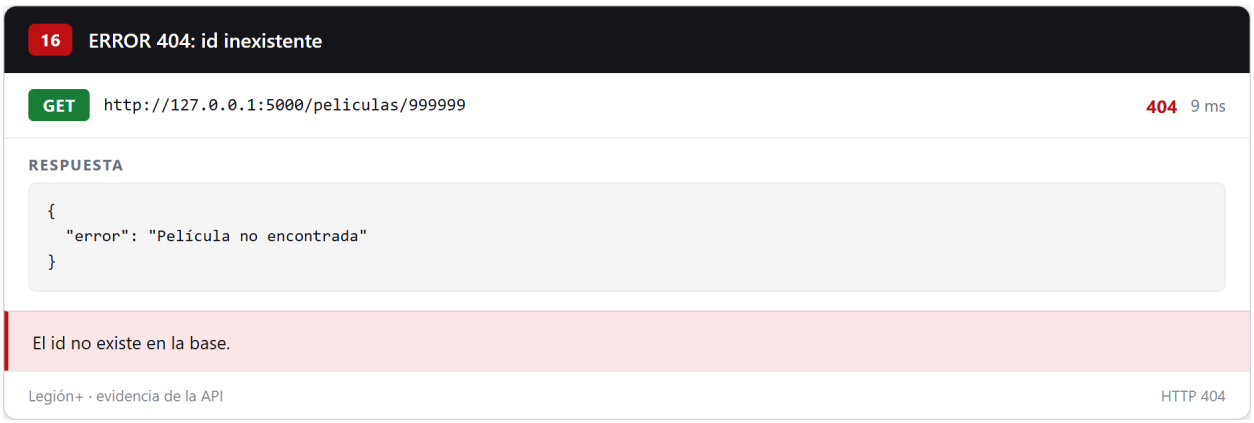


Figura 20. Error 403: un usuario intenta crear una película.

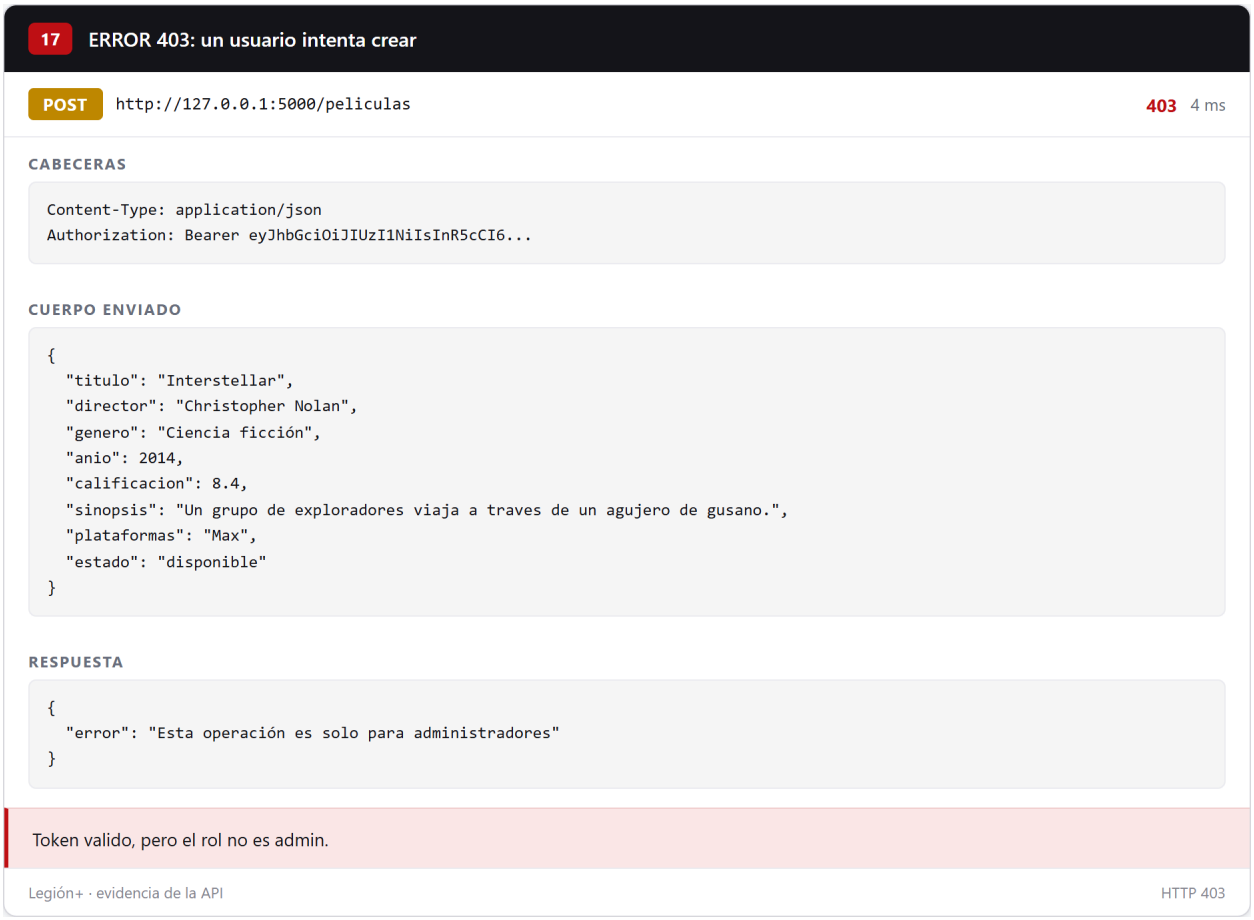


Figura 21. Error 403: un usuario intenta sincronizar con TMDB.



Figura 22. El usuario crea su propio perfil (201).

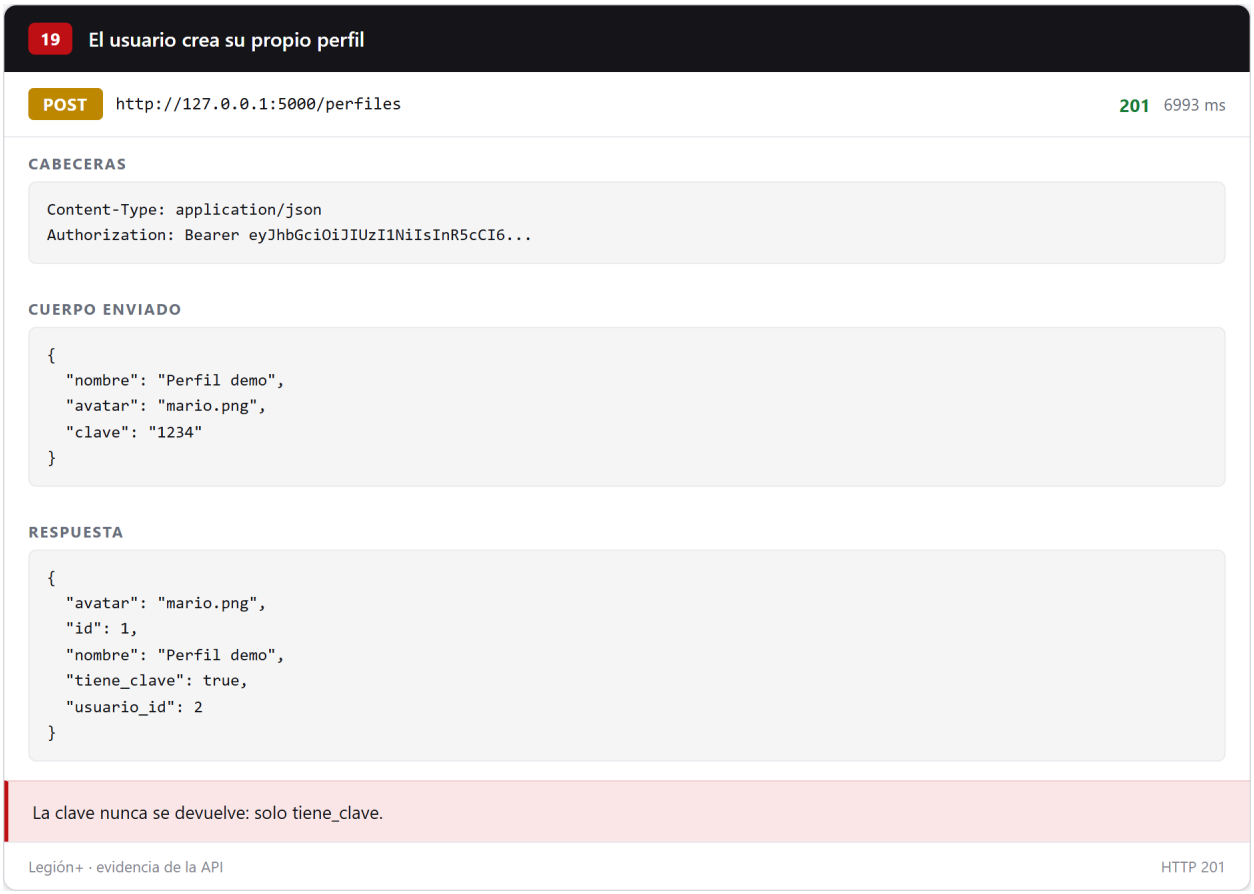


Figura 23. Registrar una segunda cuenta (201).

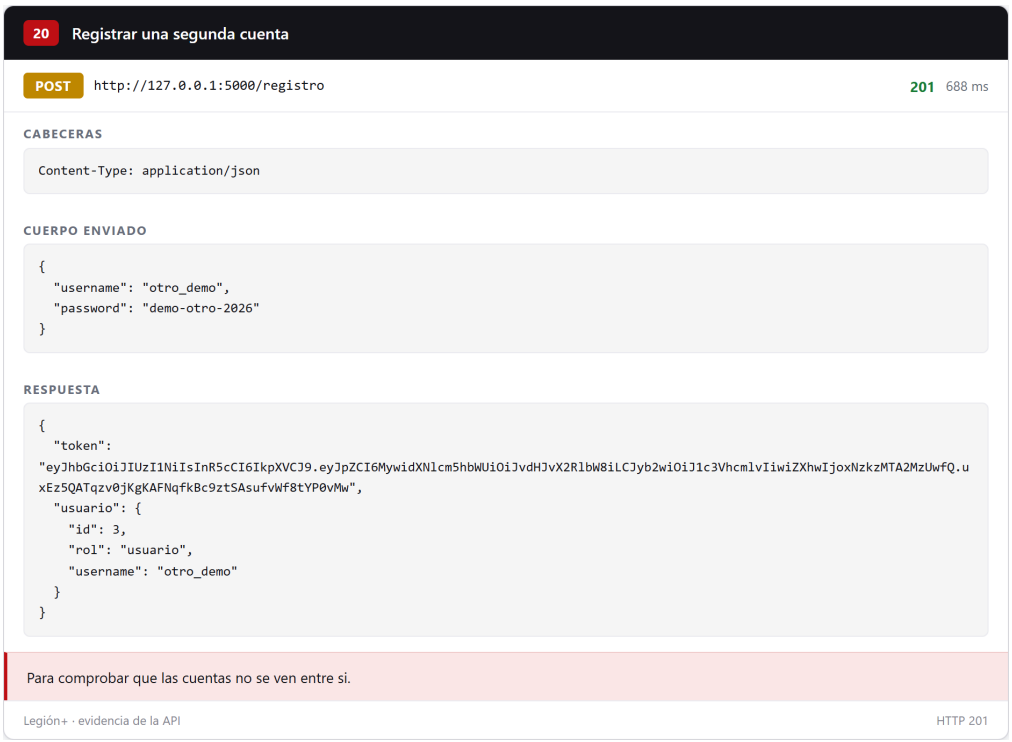


Figura 24. La otra cuenta no ve ese perfil (200).

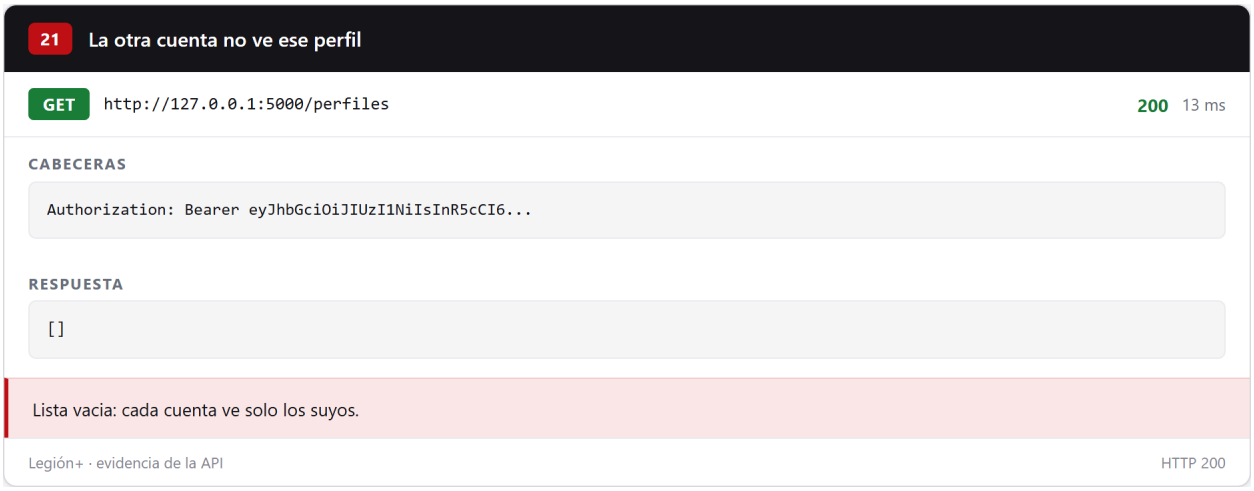


Figura 25. La otra cuenta no puede borrarlo (404).



Figura 26. Marcar una película en el perfil propio (200).

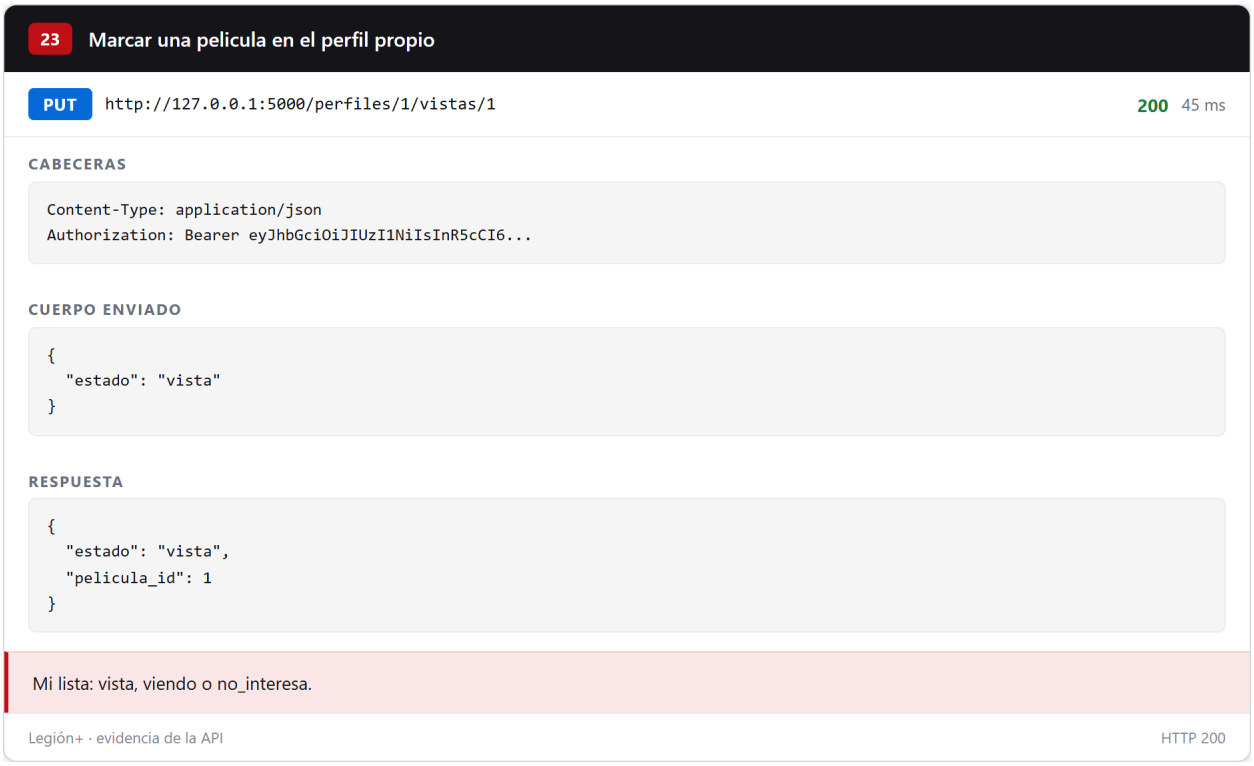


Figura 27. Consultar la lista de ese perfil (200).



Figura 28. Error 405 por método no permitido.

